MODULE 5

SD 3217 – Software Engineering II

Question

What is a system?

Definition

A system is a purposeful collection of interrelated components of different kinds that work together to deliver a set of services to the system owner and its users

System Categories

Technical computer-based systems

These are systems that include hardware and software components but not procedures and processes.

Examples: televisions, mobile phones, and other equipment with embedded software.

Sociotechnical systems

They have defined operational processes, and people (the operators) are essential parts of the system.

Example: book was created through a sociotechnical publishing system that includes various processes (creation, editing, layout, etc.) and technical systems (Microsoft Word and Excel, Adobe Illustrator, Indesign, etc.).

Four Overlapping Stages

01 Conceptual Design

02 Procurement or Acquisition

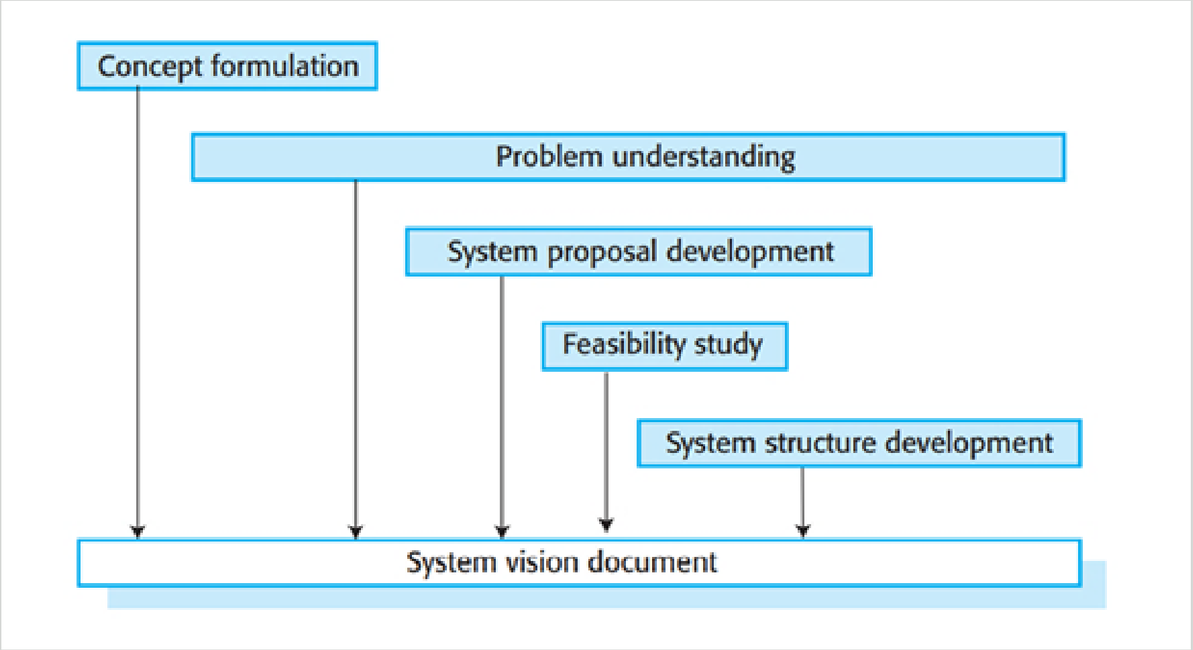
03 Development

04 Operation

First Stage

Conceptual Design

This initial system engineering activity develops the concept of the type of system that is required. It sets out, in nontechnical language, the purpose of the system, why it is needed, and the high-level features that users might expect to see in the system.



Tips to Understand a Problem

Tip 1

Discuss with users and other stakeholders how they do their work;

Tip 2

Find out what is important to them, what are the barriers that stop them from doing what they want to do, and their ideas of what changes are required;

Tip 3

Be open-minded (it is their problem, not yours); and

Tip 4

Be prepared to change your ideas when the reality does not match your initial vision.

Second Stage

Procurement or Acquisition

During this stage, the conceptual design is further developed so that information is available to make decisions about the contract for the system development. This may involve making decisions about the distribution of functionality across hardware, software, and operational processes. You also make decisions about which hardware and software has to be acquired, which suppliers should develop the system, and the terms and conditions of the supply contract.

What may be procured?

01

Off-the-shelf applications that may be used without change and that need only minimal configuration for use.

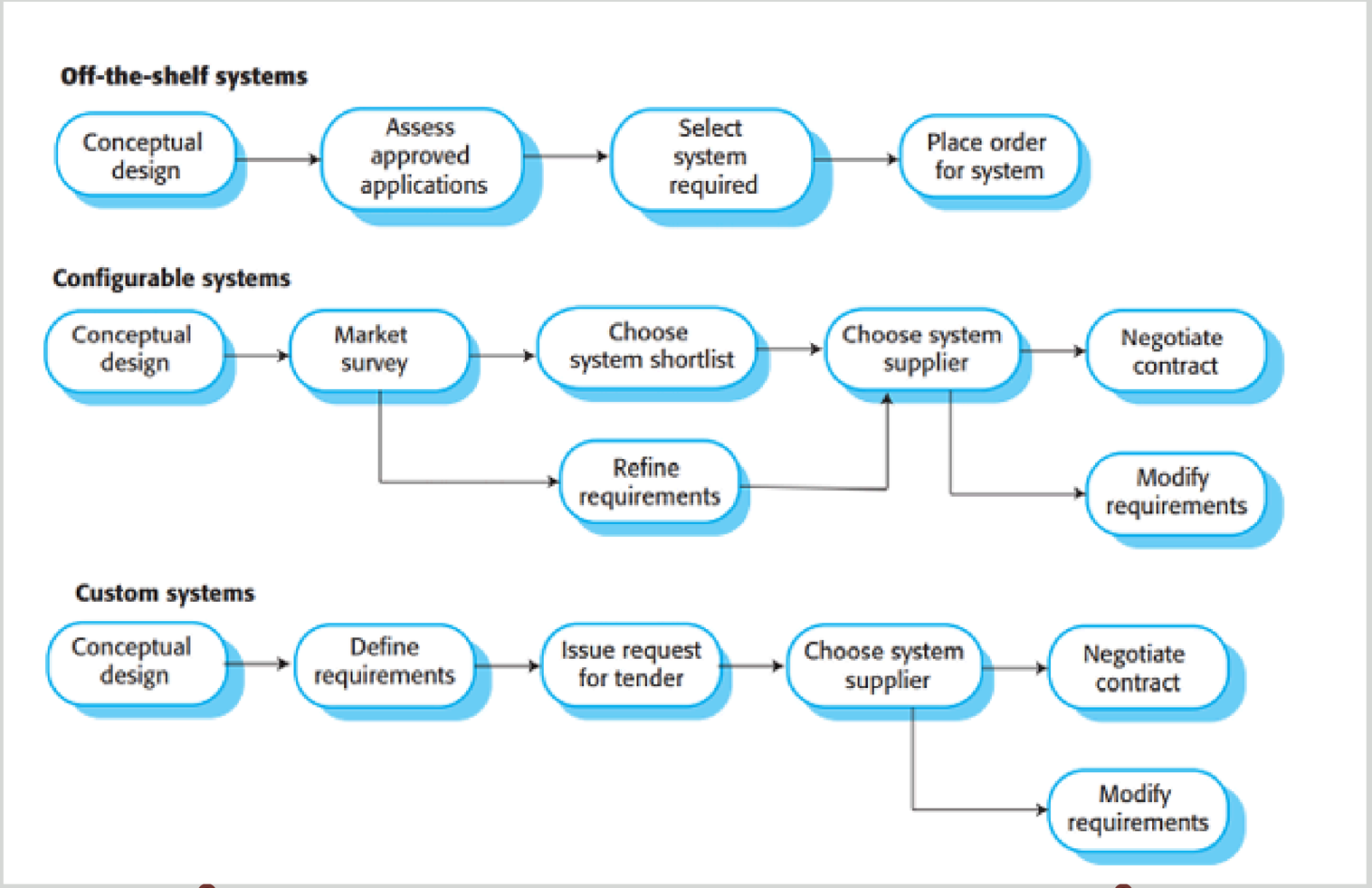
02

Configurable application or ERP systems that have to be modified or adapted for use either by modifying the code or by using inbuilt configuration features, such as process definitions and rules.

03

Custom systems that have to be specially designed and implemented for use. Each of these components tends to follow a different procurement process.

How to procure?

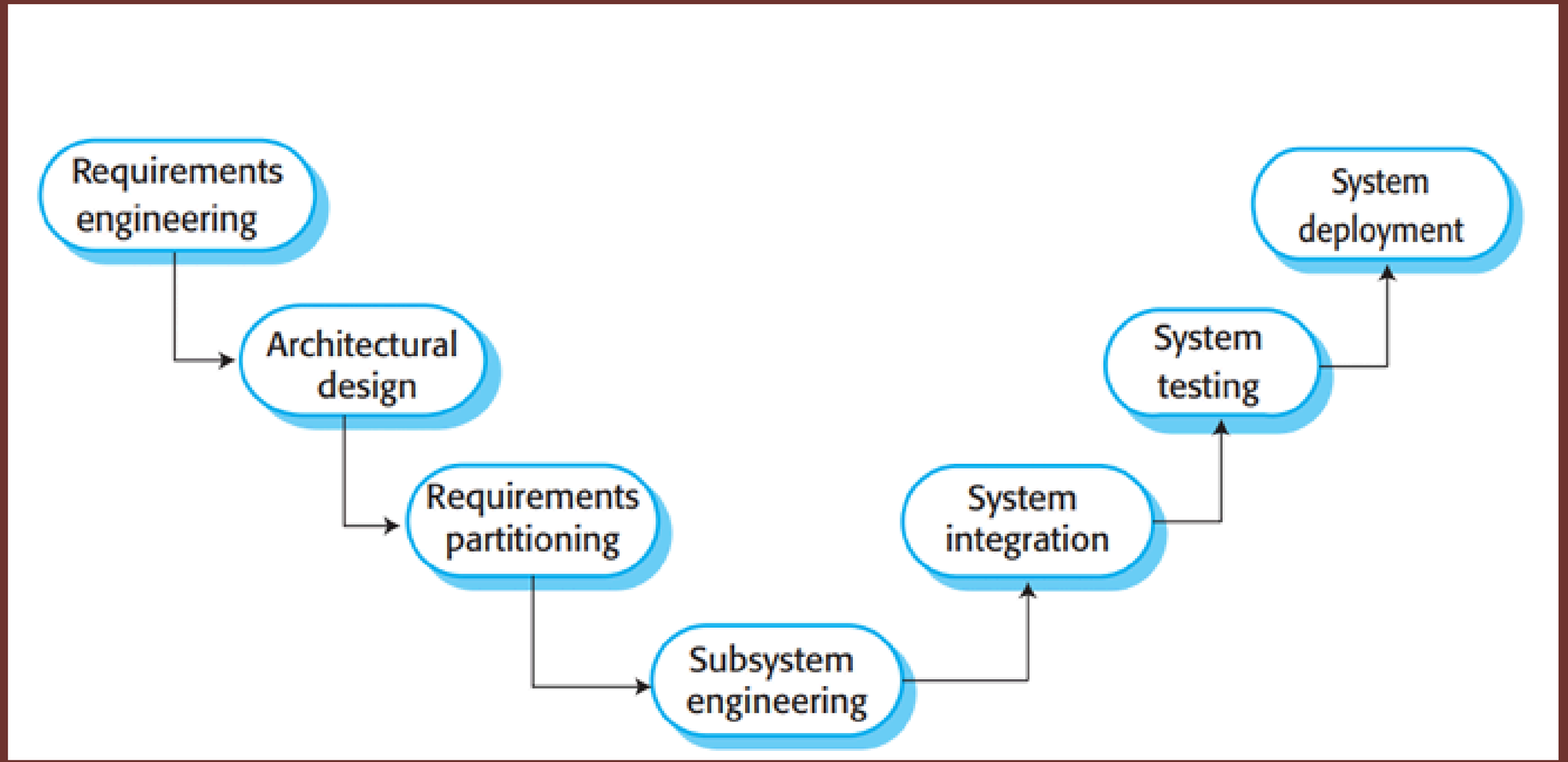


Third Stage

Development

During this stage, the system is developed. Development processes include requirements definition, system design, hardware and software engineering, system integration, and testing. Operational processes are defined, and the training courses for system users are designed.

Development activities



Fourth Stage

Operation

At this stage, the system is deployed, users are trained, and the system is brought into use. The planned operational processes usually then have to change to reflect the real working environment where the system is used. Over time, the system evolves as new requirements are identified.

System Lifetime Factors

Factor	Rationale
Investment cost	The costs of a systems engineering project may be tens or even hundreds of millions of dollars. These costs can only be justified if the system can deliver value to an organization for many years.
Loss of expertise	As businesses change and restructure to focus on their core activities, they often lose engineering expertise. This may mean that they lack the ability to specify the requirements for a new system.
Replacement cost	The cost of replacing a large system is very high. Replacing an existing system can be justified only if this leads to significant cost savings over the existing system.
Return on investment	If a fixed budget is available for systems engineering, spending on new systems in some other area of the business may lead to a higher return on investment than replacing an existing system.
Risks of change	Systems are an inherent part of business operations, and the risks of replacing existing systems with new systems cannot be justified. The danger with a new system is that things can go wrong in the hardware, software, and operational processes. The potential costs of these problems for the business may be so high that they cannot take the risk of system replacement.
System dependencies	Systems are interdependent and replacing one of these systems may lead to extensive changes in other systems.

System Evolution

System evolution, like software evolution, is inherently costly for several reasons:

Reason 01

Proposed changes have to be analyzed very carefully from a business and a technical perspective. Changes have to contribute to the goals of the system and should not simply be technically motivated.

Reason 02

Because subsystems are never completely independent, changes to one subsystem may have side-effects that adversely affect the performance or behavior of other subsystems. Consequent changes to these subsystems may therefore be needed.

System Evolution

System evolution, like software evolution, is inherently costly for several reasons:

Reason 03

The reasons for original design decisions are often unrecorded. Those responsible for the system evolution have to work out why particular design decisions were made.

Reason 04

As systems age, their structure becomes corrupted by change, so the costs of making further changes increases.

Definition

Sociotechnical System

- Sociotechnical systems are enterprise systems that are intended to help deliver a business purpose.

- A socio-technical system is one that considers requirements spanning hardware, software, personal, and community aspects. It applies an understanding of the social structures, roles and rights (the social sciences) to inform the design of systems that involve communities of people and technology.

Organizational Elements

Process Changes

A new system may mean that people have to change the way that they work. If so, training will certainly be required.

Job Changes

New systems may deskill the users in an environment or cause them to change the way they work.

Policies

The proposed system may not be completely consistent with organizational policies

Politics

The system may change the political power structure in an organization.

Characteristics

01 Emergent Properties

They have emergent properties that are properties of the system as a whole, rather than associated with individual parts of the system. Emergent properties depend on both the system components and the relationships between them.

*Functional emergent properties, when the purpose of a system only emerges after its components are integrated.

For example, a bicycle has the functional property of being a transportation device once it has been assembled from its components.

*Non-functional emergent properties, which relate to the behavior of the system in its operational environment.

Reliability, performance, safety, and security are examples of these properties. These system characteristics are critical for computer-based systems

Characteristics

02 Non-deterministic

Sociotechnical systems are nondeterministic partly because they include people and partly because changes to the hardware, software, and data in these systems are so frequent.

03 Success Criteria

The system’s success criteria are subjective rather than objective. The extent to which the system supports organizational objectives does not just depend on the system itself.

THANK YOU!

MODULE 6

SD 3217 – Software Engineering II

Definition

A system of systems (SoS) is the collection of multiple, independent systems in context as part of a larger, more complex system. A system is a group of interacting, interrelated and interdependent components that form a complex and unified whole.

SoS Characteristics

01

Operational independence of elements - Parts of the system are not simply components but can operate as useful systems in their own right.

02

Managerial independence of elements - Parts of the system are “owned” and managed by different organizations or by different parts of a larger organization. Therefore, different rules and policies apply to the management and evolution of these systems.

SoS Characteristics

03

Evolutionary development - SoS are not developed in a single project but evolve over time from their constituent systems.

04

Emergence - SoS have emergent characteristics that only become apparent after the SoS has been created. Emergence is a characteristic of all systems, but it is particularly important in SoS.

SoS Characteristics

05

Geographical distribution of elements - the elements of a SoS are often geographically distributed across different organizations. This is important technically because it means that an externally-managed network is an integral part of the SoS.

Additional

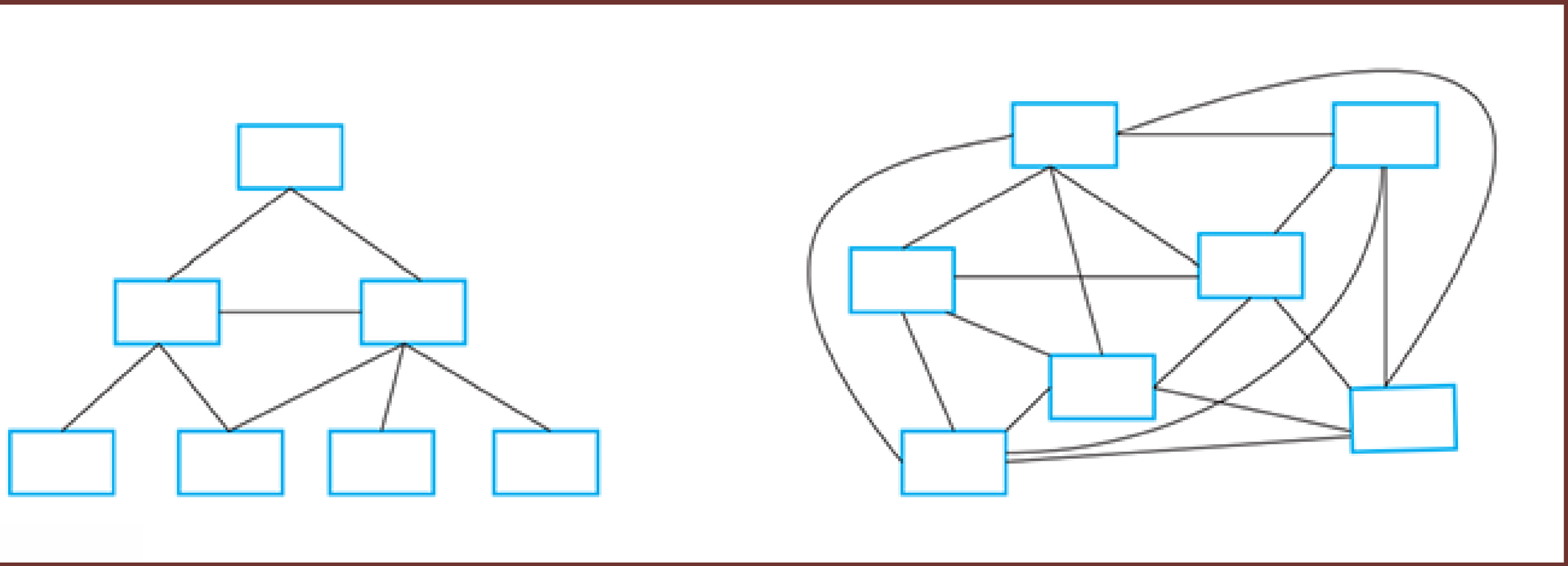
01

Data Intensive - A software SoS typically relies on and manages a very large volume of data. In terms of size, this may be tens or even hundreds of times larger than the code of the constituent systems itself.

02

Heterogeneity - The different systems in a software SoS are unlikely to have been developed using the same programming languages and design methods. This is a consequence of the very rapid pace of evolution of software technologies.

Complexity



Complexity

Static relationship

Static relationships are relationships that are planned and analyzable from static depictions of the system. From either the software source code or a UML model of a system, you can work out how anyone software component uses other components

Dynamic relationship

Dynamic relationships are relationships that exist in an executing system. Dynamic relationships are more complex to analyze as you need to know the system inputs and data used as well as the source code of the system.

Complexity

01

The technical complexity of the system is derived from the relationships between the different components of the system itself.

02

The managerial complexity of the system is derived from the complexity of the relationships between the system and its managers and the relationships between the managers of different parts of the system.

03

The governance complexity of a system depends on the relationships between the laws, regulations, and policies that affect the system and the relationships between the decision making processes in the organizations responsible for the system.

SoS Classification

01

Directed SoS are owned by a single organization and are developed by integrating systems that are also owned by that organization.

02

Collaborative SoS are systems with no central authority to set management priorities and resolve disputes. Typically, elements of the system are owned and governed by different organizations.

03

Virtual systems have no central governance, and the participants may not agree on the overall purpose of the system. Participant systems may enter or leave the SoS.

Definition

Reductionism

This notion is the foundation for the engineering of large systems – essentially, design a system so that it is composed of discrete parts which, because they are smaller, are easier to understand and construct, define interfaces that allow these parts to work together, build the parts of the system then integrate these to create the desired system.

Assumptions

01 System Ownership and Control

Reductionism assumes that there is a controlling authority for a system that can resolve disputes and make high-level technical decisions that will apply across the system.

02 Rational Decision-Making

Reductionism assumes that interactions between components can be objectively assessed by, for example, mathematical modeling. These assessments are the driver for system decision making.

Assumptions

03 Defined System Boundaries

Reductionism assumes that the boundaries of a system can be agreed to and defined.

Definition

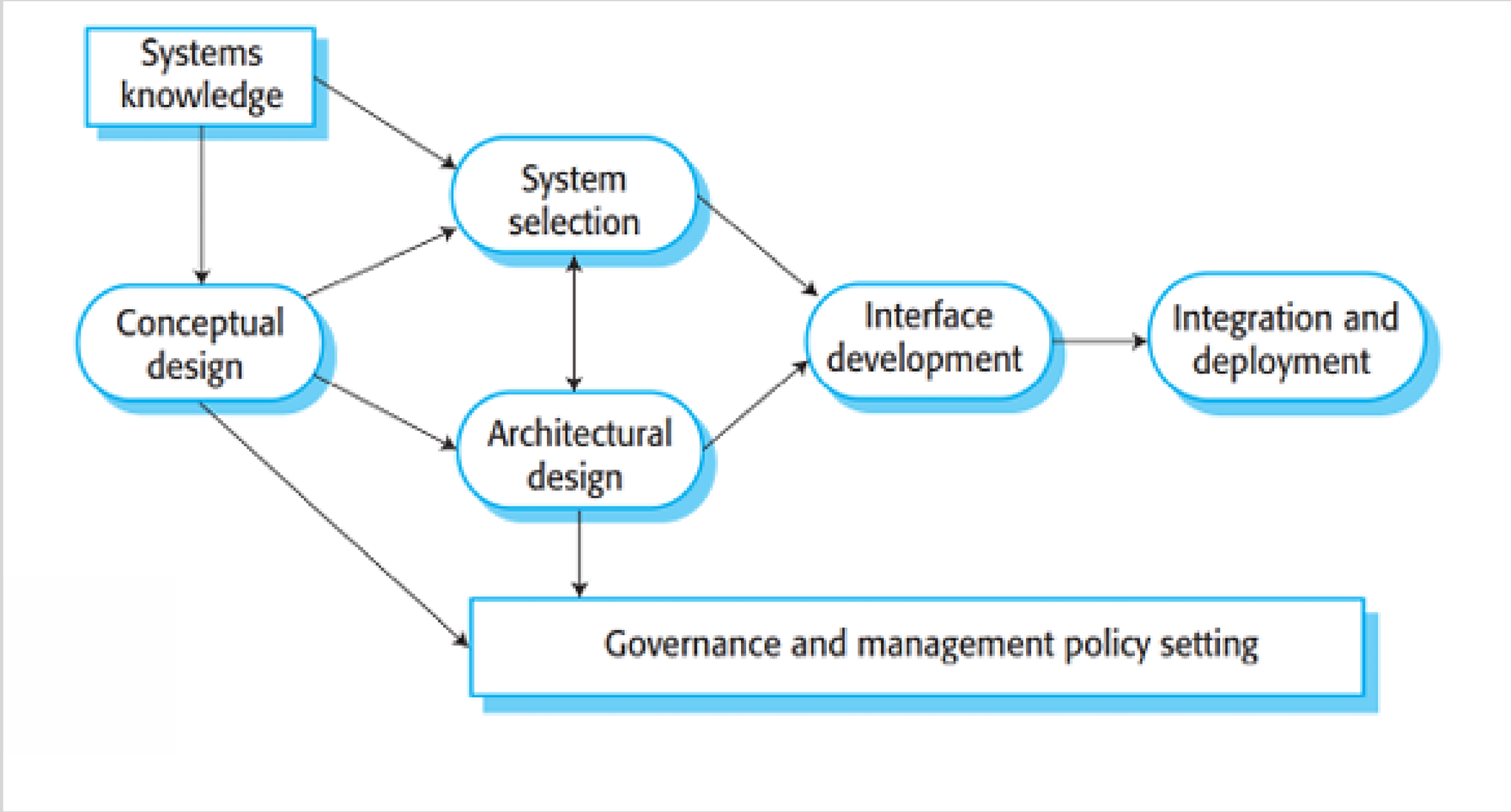
Systems of Systems Engineering

Systems of systems engineering is the process of integrating existing systems to create new functionality and capabilities.

The problems of software SoS engineering has much in common with the problems of integrating large-scale application systems.

1. Lack of control over system functionality and performance.
2. Differing and incompatible assumptions made by the developers of the different systems.
3. Different evolution strategies and timetables for the different systems.
4. Lack of support from system owners when problems arise.

SoS Engineering Processes



Stages of Deployment Process

First

The initial deployment provides authentication, basic learning functionality, and integration with school administration systems.

Second

The second stage adds an integrated storage system and a set of more specialized tools to support subject-specific learning.

Third

Stage 3 adds features for user configuration and the ability of users to add new systems to the environment.

Testing SoS

Testing systems of systems is difficult and expensive for three reasons:

Reason 01

There may not be a detailed requirements specification that can be used as a basis for system testing.

Reason 02

The constituent systems may change in the course of the testing process, so tests may not be repeatable.

Testing SoS

Testing systems of systems is difficult and expensive for three reasons:

Reason 03

If problems are discovered, it may not be possible to fix the problems by requiring one or more of the constituent systems to be changed. Rather, some intermediate software may have to be introduced to solve the problem.

Agile Testing

01

Agile methods do not rely on having a complete system specification for system acceptance testing. Rather, stakeholders are closely engaged with the testing process and have the authority to decide when the overall system is acceptable.

02

Agile methods make extensive use of automated testing. This makes it much easier to rerun tests to discover if unexpected system changes have caused problems for the SoS as a whole.

Definition

SoS Architecture

Perhaps the most crucial activity of the systems of systems engineering process is architectural design. Architectural design involves selecting the systems to be included in the SoS, assessing how these systems will interoperate, and designing mechanisms that facilitate interaction.

Architecture Principles

01

Design systems so that they can deliver value even if they are incomplete.

02

Be realistic about what can be controlled.

03

Focus on the system interfaces.

04

Provide collaboration incentives.

Additional Principles

01 Design a SoS as node and web architecture.

02 Specify behavior as services exchanged between nodes.

03 Understand and manage system vulnerabilities.

THANK YOU!

MODULE 7

SD 3217 – Software Engineering II

Definition

Embedded systems

- Embedded systems are purpose-built computers integrated into products, delivering specific functions rather than general computing capabilities, from household electronics to complex transportation systems.
- They operate under tight constraints, including limited processing power, memory, storage, and energy, while often needing to meet strict real-time deadlines and reliability expectations.

Characteristics

01

Embedded systems generally run continuously and do not terminate.

02

Interactions with the system’s environment are unpredictable.

Characteristics

03

Physical limitations may affect the design of a system.

04

Direct hardware interaction may be necessary.

05

Issues of safety and reliability may dominate the system design.

Stimuli

Stimulus	Response
Clear alarms	Switch off all active alarms; switch off all lights that have been switched on.
Console panic button positive	Initiate alarm; turn on lights around console; call police.
Power supply failure	Call service technician.
Sensor failure	Call service technician.
Single sensor positive	Initiate alarm; turn on lights around site of positive sensor.
Two or more sensors positive	Initiate alarm; turn on lights around sites of positive sensors; call police with location of suspected break-in.
Voltage drop of between 10% and 20%	Switch to battery backup; run power supply test.
Voltage drop of more than 20%	Switch to battery backup; initiate alarm; call police, run power supply test.

Stimuli Classes

Periodic Stimuli

These occur at predictable time intervals. For example, the system may examine a sensor every 50 milliseconds and take action (respond) depending on that sensor value (the stimulus).

Aperiodic Stimuli

These occur irregularly and unpredictably and are usually signaled, using the computer's interrupt mechanism. An example of such a stimulus would be an interrupt indicating that an I/O transfer was complete and that data was available in a buffer.

Design Process

01

Platform Selection - In this activity, you choose an execution platform for the system, that is, the hardware and the real-time operating system to be used.

02

Stimuli/Response Identification - This involves identifying the stimuli that the system must process and the associated response or responses for each stimulus

03

Timing Analysis - For each stimulus and associated response, you identify the timing constraints that apply to both stimulus and response processing.

Design Process

04

Process Design - Process design involves aggregating the stimulus and response processing into a number of concurrent processes.

05

Algorithm Design - For each stimulus and response, you design algorithms to carry out the required computations.

06

Data Design- You specify the information that is exchanged by processes and the events that coordinate information exchange, and design data structures to manage this information exchange.

Design Process

07

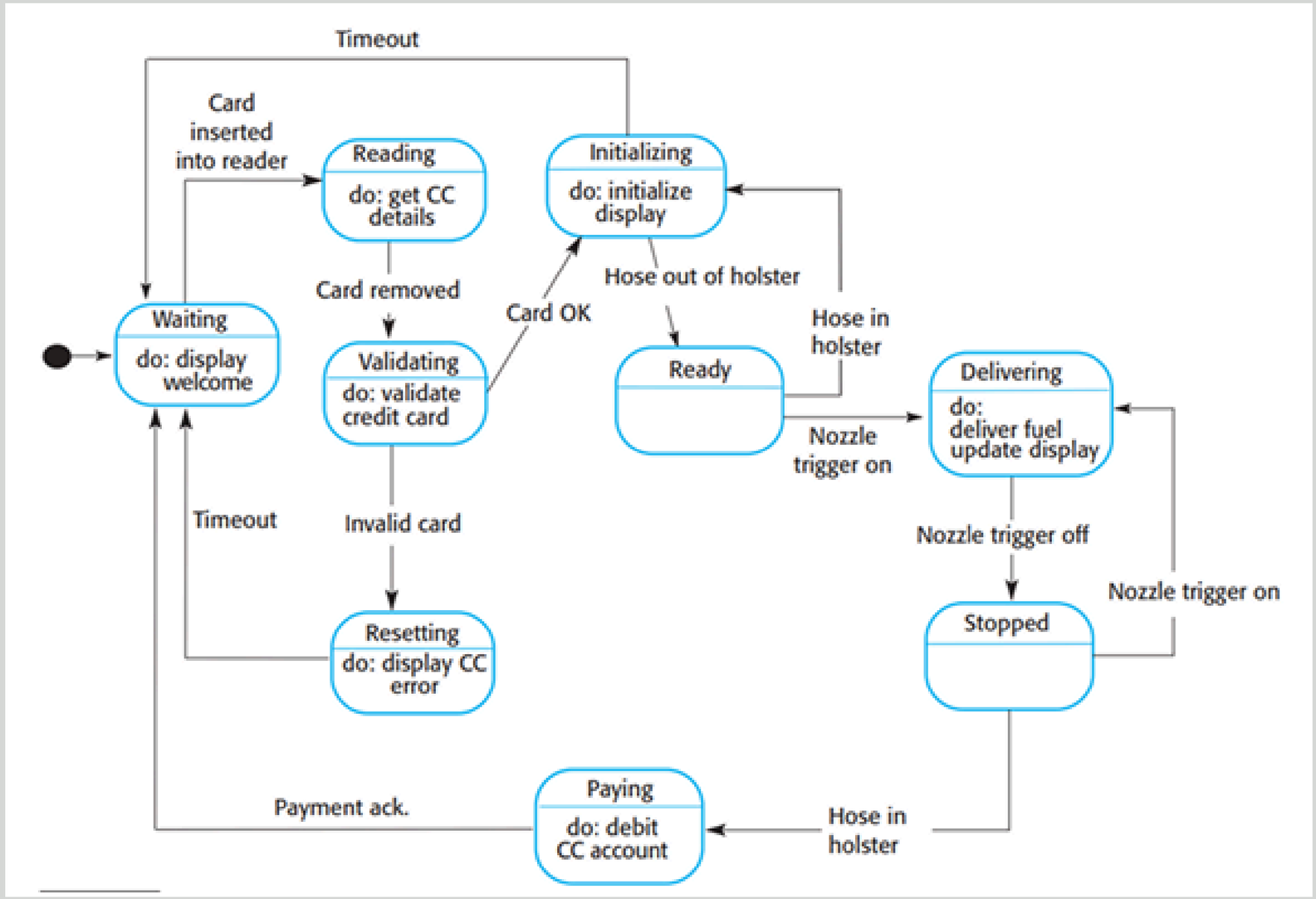
Process Scheduling - You design a scheduling system that will ensure that processes are started in time to meet their deadlines.

Definition

State Machine Model/Diagram

- It is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions.
- A state machine diagram is used to model the dynamic behavior of a class in response to time and changing external stimuli(events that cause the system to change its state from one to another).

Example


Timeout
Card inserted into reader
Reading
do: get CC details
Initializing
do: initialize display
Card removed
Hose out of holster
Card OK
Waiting
do: display welcome
Validating
do: validate credit card
Ready
Hose in holster
Delivering
do: deliver fuel update display
Nozzle trigger on
Timeout
Invalid card
Nozzle trigger off
Nozzle trigger on
Resetting
do: display CC error
Stopped
Paying
do: debit CC account
Payment ack.
Hose in holster

Architectural Patterns

01

Observe and React - This pattern is used when a set of sensors are routinely monitored and displayed.

02

Environmental Control - This pattern is used when a system includes sensors, which provide information about the environment and actuators that can change the environment.

03

Process Pipeline - This pattern is used when data has to be transformed from one representation to another before it can be processed.

Definition

Timing Analysis

The correctness of a real-time system depends not just on the correctness of its outputs but also on the time at which these outputs were produced. Therefore, timing analysis is an important activity in the embedded, real-time software development process. In such an analysis, you calculate how often each process in the system must be executed to ensure that all inputs are processed and all system responses are produced in a timely way.

Three key factors

01

Deadlines- The times by which stimuli must be processed and some response produced by the system.

02

Frequency- The number of times per second that a process must execute so that you are confident that it can always meet its deadlines.

03

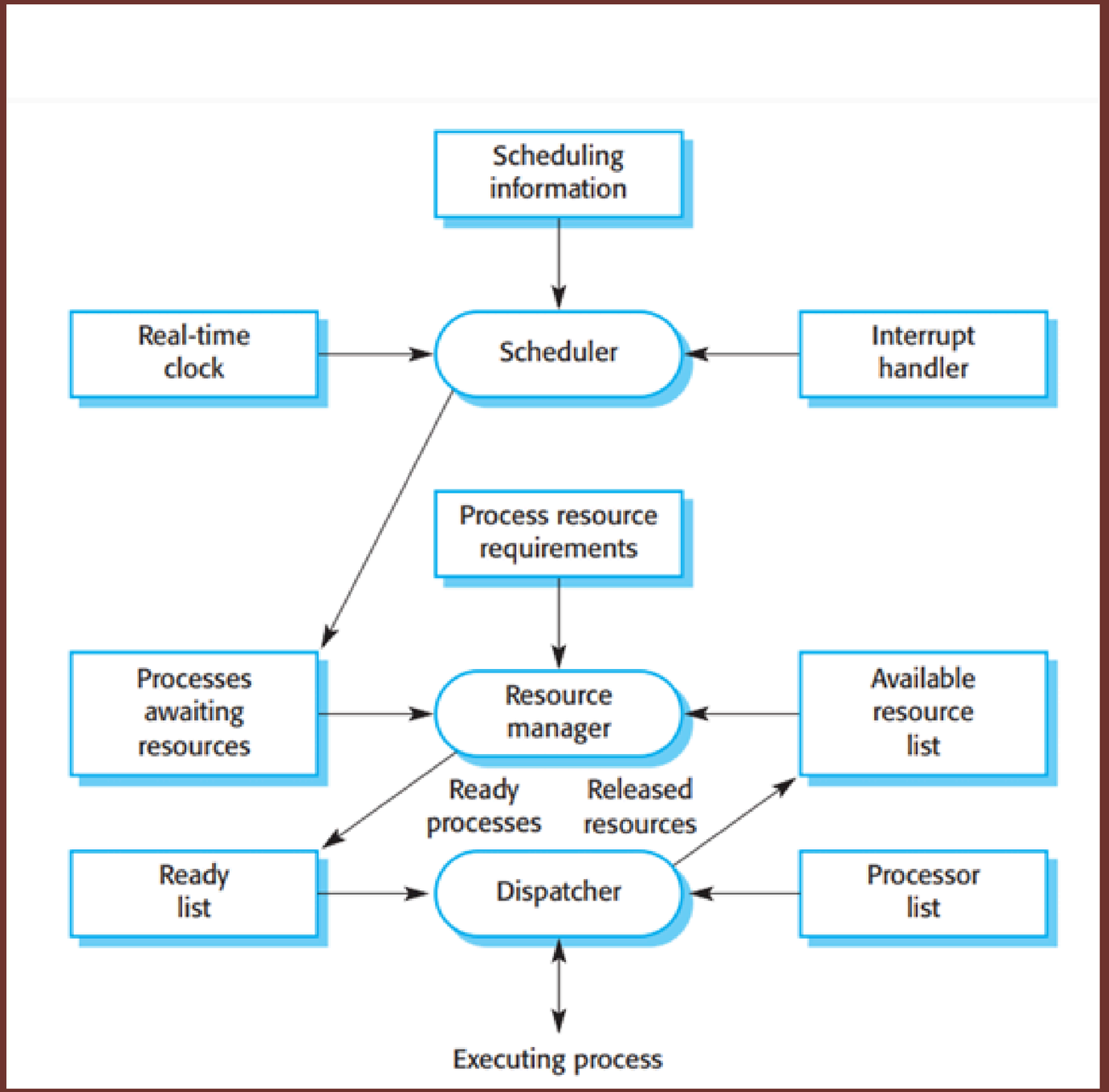
Execution Time - The time required to process a stimulus and produce a response. Execution time is not always the same because of the conditional execution of code, delays waiting for other processes, and so on.

Definition

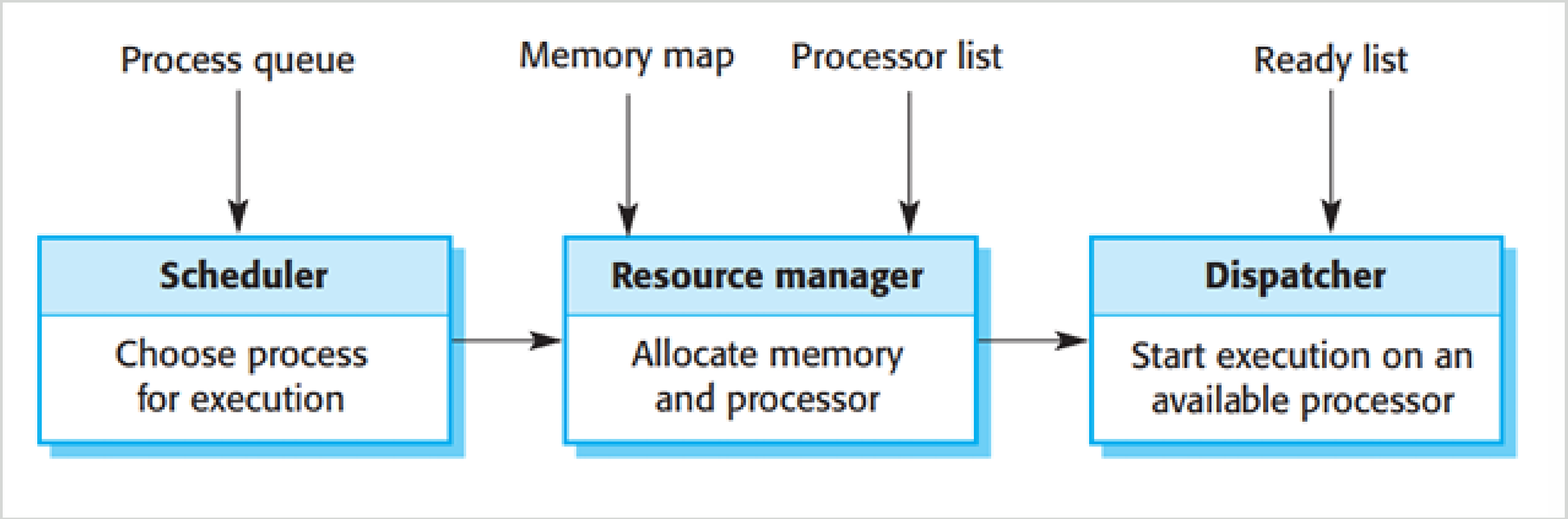
Real-time Operating System

More commonly, embedded applications are built on top of a real-time operating system (RTOS), which is an efficient operating system that offers the features needed by real-time systems. Examples of RTOS are Windows Embedded Compact, VxWorks, and RTLinux.

RTOS


Scheduling information
Real-time clock
Scheduler
Interrupt handler
Process resource requirements
Processes awaiting resources
Resource manager
Available resource list
Ready processes
Released resources
Ready list
Dispatcher
Processor list
Executing process

RTOS Actions



Scheduling Strategy

01

Non-preemptive Scheduling - After a process has been scheduled for execution, it runs to completion or until it is blocked for some reason, such as waiting for input.

02

Preemptive Scheduling - The execution of an executing process may be stopped if a higher-priority process requires service. The higher-priority process preempts the execution of the lower-priority process and is allocated to a processor.

THANK YOU!